



TEMA 7

Funções em JavaScript



Conceitos



10 questões



function | arrow | closures | DOM

O que são Funções?

 **Analogia:** Funções são como **receitas de bolo**. Você tem uma lista de instruções (a função) que pode ser executada sempre que quiser fazer um bolo. Cada vez que você segue a receita, o resultado é o mesmo!

Funções são blocos de código que executam uma tarefa específica e podem retornar um resultado. Elas são usadas para **organizar**, **reutilizar** e **modularizar** o código.

 **Exemplo do dia a dia:** Quando você usa uma calculadora, cada botão (soma, subtração, multiplicação) é como uma função. Você passa os números e a calculadora devolve o resultado.

Por que usar funções?



Reutilização

Escreva uma vez, use várias vezes



Organização

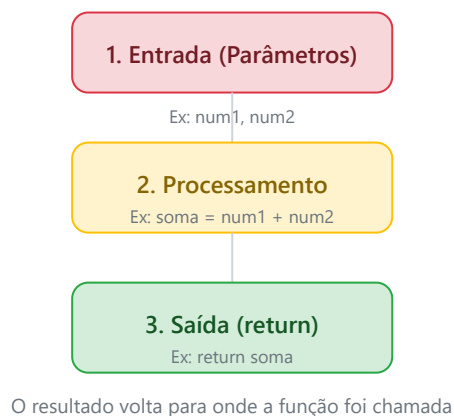
Código mais limpo e legível



Manutenção

Facilita correção de bugs

Como uma função funciona?



Fluxograma: Entrada → Processamento → Saída

✓ Resumo:

- **Entrada:** A função recebe dados (parâmetros)
- **Processamento:** A função faz algo com esses dados

- **Saída:** A função devolve um resultado (return)



Partes de uma Função

Para entender bem as funções, vamos analisar cada parte da sua estrutura:

```
// ESTRUTURA DE UMA FUNÇÃO

function nomeDaFuncao(parametro1, parametro2) {
  // 1. NOME: identifica a função (ex: soma, dobro, etc)
  // 2. PARÂMETROS: variáveis que recebem os valores de entrada
  // 3. CORPO: as instruções que a função executa
  let resultado = parametro1 + parametro2;
  return resultado; // 4. RETORNO: o valor que a função devolve
}
```

1. Nome da Função

Deve descrever o que a função faz. Use verbos no infinitivo: `somar()`, `calcular()`, `exibir()`

2. Parâmetros

Valores que a função recebe para processar. Ficam entre parênteses `()`

3. Corpo da Função

O bloco de código entre chaves `{ }` que executa a tarefa

4. Retorno (return)

O valor que a função devolve. Se não houver `return`, retorna `undefined`



Dica importante: Quando a função encontra a palavra `return`, ela **para imediatamente** e devolve o valor. Nada depois do `return` será executado!

Tipos de Funções em JavaScript

Existem várias maneiras de criar funções em JavaScript. Cada uma tem suas características e usos específicos. Vamos conhecer todas!

1. Função Nomeada

 **O que é?** É a forma mais tradicional de criar uma função. Ela tem um nome e pode ser chamada em qualquer lugar do código.

✓ Como escrever

```
function soma(a, b) {  
    return a + b;  
}
```

💡 Como usar

```
let resultado = soma(3, 5);  
console.log(resultado); // 8
```

✓ Vantagens:

- **Reutilizável:** Pode ser chamada várias vezes
- **Organizada:** O código fica mais legível
- **Modular:** Separa tarefas em blocos
- **Auto-documentada:** O nome explica o que faz

exemplo-funcao-nomeada.js

```
// =====  
// FUNÇÃO NOMEADA - A forma mais comum de criar funções  
// =====  
  
// 1. Função que calcula o dobro de um número  
function calcularDobro(numero) {  
    return numero * 2;  
}  
  
// 2. Função que verifica se um número é par  
function ehPar(numero) {
```

```
if (numero % 2 === 0) {  
    return "✅ É par";  
} else {  
    return "❌ É ímpar";  
}  
}
```

// 3. Usando as funções

🚀 2. Função Anônima

📖 **O que é?** É uma função que **não tem nome**. Ela é atribuída a uma variável ou passada como argumento para outra função.

✅ Como escrever

```
const soma = function(a, b) {  
    return a + b;  
};
```

💡 Como usar

```
let resultado = soma(3, 5);  
console.log(resultado); // 8
```

✅ Quando usar?

- Quando a função é usada apenas **uma vez**
- Como **callback** (função passada como argumento)
- Em **eventos** (onclick, onload, etc.)

📄 exemplo-funcao-anonima.js

```
// =====  
// FUNÇÃO ANÔNIMA - Função sem nome, atribuída a uma variável  
// =====  
  
// 1. Função anônima atribuída a uma variável  
const calcularDobro = function(numero) {  
    return numero * 2;  
};  
  
// 2. Função anônima como callback (passada como argumento)
```

```
const numeros = [1, 2, 3, 4, 5];
numeros.forEach(function(item) {
    console.log(item * 2);
});

// 3. Usando a função anônima
console.log(calcularDobro(5)); // 10
```

🚩 3. Arrow Function (Função de Flecha)

📖 **O que é?** É uma **versão simplificada** da função anônima, introduzida no ES6 (2015). Usa a sintaxe `=>` (flecha).

✅ Como escrever

```
// Sem parâmetros
const ola = () => "Olá!";

// Com um parâmetro
const dobro = (x) => x * 2;

// Com vários parâmetros
const soma = (a, b) => a + b;
```

💡 Como usar

```
console.log(ola()); // Olá!
console.log(dobro(5)); // 10
console.log(soma(3, 5)); // 8
```

💡 Regras da Arrow Function:

- Se tem **apenas 1 parâmetro**, pode omitir os parênteses: `x => x * 2`
- Se o corpo tem **apenas 1 linha**, pode omitir as chaves e o `return`
- **Não tem** seu próprio `this` (herda do escopo pai)
- Não pode ser usada como **construtora** (não tem `new`)

📄 exemplo-arrow-function.js

```
// =====
// ARROW FUNCTION - Sintaxe simplificada com =>
// =====
```

```
// 1. Arrow function com vários parâmetros
const soma = (a, b) => a + b;
console.log(soma(3, 5)); // 8

// 2. Arrow function com um parâmetro (pode omitir os parênteses)
const dobro = x => x * 2;
console.log(dobro(7)); // 14

// 3. Arrow function com bloco de código (várias linhas)
const verificarNumero = (num) => {
  if (num > 0) {
    return "Positivo";
  } else if (num < 0) {
    return "Negativo";
  } else {
```

📌 4. Função Construtora

📖 **O que é?** É uma função usada para **criar objetos** com a palavra-chave `new`. Por convenção, seu nome começa com letra maiúscula.

✅ Como escrever

```
function Pessoa(nome, idade) {
  this.nome = nome;
  this.idade = idade;
}
```

💡 Como usar

```
const ana = new Pessoa("Ana", 25);
console.log(ana.nome); // Ana
console.log(ana.idade); // 25
```

💡 O que é o `this` ?

O `this` em uma função construtora se refere ao **objeto que está sendo criado**. Quando usamos `this.nome = nome`, estamos dizendo: "No objeto que estou criando, crie uma propriedade chamada `nome` com o valor que foi passado como parâmetro".

📄 exemplo-funcao-construtora.js

```
// =====  
// FUNÇÃO CONSTRUTORA - Cria objetos com a palavra 'new'  
// =====  
  
// 1. Definindo a função construtora (nome com letra maiúscula)  
function Carro(marca, modelo, ano) {  
    this.marca = marca;  
    this.modelo = modelo;  
    this.ano = ano;  
    this.ligar = function() {  
        return "🚗 " + this.modelo + " está ligado!";  
    };  
}  
  
// 2. Criando objetos a partir da função construtora  
const carro1 = new Carro("Toyota", "Corolla", 2020);  
const carro2 = new Carro("Honda", "Civic", 2022);  
  
// 3. Usando os objetos criados
```

🚀 5. Closure (Fechamento)

📖 **O que é?** Uma **função que "lembra"** das variáveis do escopo onde foi criada, mesmo depois que a função externa terminou.

✅ Como escrever

```
function contador() {  
    let count = 0;  
    return function() {  
        count++;  
        return count;  
    };  
}
```

💡 Como usar

```
const incrementar = contador();  
console.log(incrementar()); // 1  
console.log(incrementar()); // 2  
console.log(incrementar()); // 3
```

💡 Por que isso é útil?

- Permite criar **variáveis privadas** (não acessíveis fora da closure)
- Útil para **contadores**, **gerenciamento de estado**
- Muito usado em **eventos** e **callbacks**

exemplo-closure.js

```
// =====  
// CLOSURE - Função que "lembra" do escopo onde foi criada  
// =====  
  
// 1. Exemplo de Closure - Contador  
function criarContador() {  
    let count = 0; // Esta variável é "lembrada" pela closure  
  
    return function() {  
        count++;  
        return count;  
    };  
}  
  
const contador1 = criarContador();  
const contador2 = criarContador();  
  
console.log(contador1()); // 1  
console.log(contador1()); // 2
```

6. Eventos e DOM

 **O que é o DOM?** O **DOM (Document Object Model)** é a **representação da página HTML** como uma árvore de objetos. O JavaScript pode **manipular** essa árvore para alterar a página dinamicamente.

Como acessar elementos

```
// Acessar por ID  
let elemento = document.getElementById("meuId");  
  
// Acessar por classe  
let itens = document.getElementsByClassName("minhaClasse");  
  
// Acessar por seletor CSS  
let elemento = document.querySelector(".classe");
```

✓ Como adicionar eventos

```
// Método moderno (recomendado)
elemento.addEventListener("click", function() {
    // código executado ao clicar
});

// Método tradicional
elemento.onclick = function() {
    // código executado ao clicar
};
```

✓ Principais eventos:

- `click` - quando o usuário clica em um elemento
- `mouseover` - quando o mouse passa sobre o elemento
- `mouseout` - quando o mouse sai do elemento
- `keydown` - quando uma tecla é pressionada
- `keyup` - quando uma tecla é solta
- `change` - quando um campo de formulário é alterado
- `submit` - quando um formulário é enviado
- `load` - quando a página termina de carregar

exemplo-eventos-dom.js

```
// =====
// EVENTOS E DOM - Manipulando a página dinamicamente
// =====

// 1. Função que altera o conteúdo da página
function alterarTexto() {
    let elemento = document.getElementById("mensagem");
    elemento.textContent = "O texto foi alterado!";
}

// 2. Função que altera a cor de fundo
function alterarCor(cor) {
    document.body.style.backgroundColor = cor;
}

// 3. Adicionando evento a um elemento (simulação)
console.log("Aperte F12 para ver o console.");
console.log("Em uma página real, você usaria:");
```

```
console.log("document.getElementById('botao').onclick = alterarTexto;");
```

⚠️ **ATENÇÃO:** Arrow functions **NÃO têm** seu próprio `this`. Elas herdam o `this` do escopo pai. Use funções nomeadas ou anônimas quando precisar manipular o `this` (ex: em funções construtoras ou eventos).



Comparação entre os Tipos de Funções

Característica	Nomeada	Anônima	Arrow	Construtora
Tem nome?	✓ Sim	✗ Não	✗ Não	✓ Sim
Usa function	✓ Sim	✓ Sim	✗ Não	✓ Sim
Tem this próprio?	✓ Sim	✓ Sim	✗ Não	✓ Sim
Pode usar new?	✓ Sim	✓ Sim	✗ Não	✓ Sim
Pode ser callback?	✓ Sim	✓ Sim	✓ Sim	✗ Não
Sintaxe	Mais longa	Média	Mais curta	Média
Melhor para	Funções reutilizáveis	Callbacks	Funções curtas	Criar objetos



Resumo:

- Use **função nomeada** para tarefas principais e reutilizáveis
- Use **arrow function** para funções curtas e callbacks simples
- Use **função construtora** para criar múltiplos objetos do mesmo tipo
- Use **closure** para criar variáveis privadas e estado encapsulado

Resumo das Funções

◆ Função Nomeada

Declarada com `function` e um nome. Reutilizável e organizada.

```
function soma(a, b) {  
  return a + b;  
}
```

◆ Arrow Function

Sintaxe simplificada com `=>`. Ideal para funções curtas.

```
const soma = (a, b) => a + b;
```

◆ Função Construtora

Cria objetos com `new`. Nome começa com maiúscula.

```
function Pessoa(nome) {  
  this.nome = nome;  
}
```

◆ Closure

Função que "lembra" do escopo onde foi criada.

```
function contador() {  
  let count = 0;  
  return function() { return ++count; };  
}
```



Glossário de Termos

Função

Bloco de código que executa uma tarefa específica

Parâmetro

Valor que a função recebe para processar

return

Palavra-chave que devolve o resultado da função

Arrow Function

Sintaxe simplificada de função usando =>

Closure

Função que mantém acesso ao escopo externo

this

Referência ao objeto atual da função

DOM

Representação da página HTML como árvore de objetos

Evento

Ação do usuário (clique, tecla, mouse, etc.)

Callback

Função passada como argumento para outra função

Escopo

Região do código onde uma variável é acessível



Exercício de Fixação

Responda as **10 questões** sobre Funções em JavaScript.

Ao final, clique em "**Verificar Respostas**" para corrigir.

1 Qual palavra-chave é usada para criar uma função nomeada?

☐ const

☒ function

☐ let

☐ var

2 Qual é a sintaxe correta de uma Arrow Function?

☐ function soma(a, b) { return a + b; }

☐ const soma = function(a, b) { return a + b; }

☐ const soma = (a, b) => a + b;

☒ const soma = a, b => a + b;

3 Qual palavra-chave é usada para retornar um valor de uma função?

☒ return

☐ break

☐ continue

☐ exit

4 O que é uma Closure em JavaScript?

☐ Uma função que não retorna nada

☒ Uma função que tem acesso ao escopo de sua função externa

☐ Uma função que só pode ser usada uma vez

☐ Uma função que não tem parâmetros

5 O que faz o método `addEventListener()` ?

☐ Remove um evento

☒ Adiciona um evento a um elemento

☐ Cria um novo elemento

☐ Altera o estilo de um elemento

6 Qual evento é acionado quando o usuário clica em um elemento?

☐ `onMouseOver`

☐ `onKeyPress`

☒ `onClick`

☐ `onLoad`

7 O que a palavra-chave `this` representa em uma função construtora?

☐ O objeto global

☒ O objeto que está sendo criado

☐ A função em si

☐ O parâmetro da função

8 Arrow functions têm seu próprio `this` .

☒ Verdadeiro

☐ Falso

9

O DOM (Document Object Model) representa a estrutura da página como uma árvore de nós.

☒ Verdadeiro

☐ Falso

10 Qual método é usado para acessar um elemento pelo seu ID?

☐ document.querySelector()

☒ document.getElementById()

☐ document.getElementsByClassName()

☐ document.getElementsByTagName()



Resultado do Quiz

9 / 10

Respostas corretas:

- ✓ Q1 - Correta
function é a palavra-chave usada para declarar funções nomeadas.
- ✗ Q2 - Incorreta
Arrow functions usam a sintaxe (param) => expressão.
- ✓ Q3 - Correta
return é usado para retornar um valor da função.
- ✓ Q4 - Correta
Closure é uma função que mantém acesso ao escopo da função externa.
- ✓ Q5 - Correta
addEventListener() é usado para adicionar eventos a elementos HTML.
- ✓ Q6 - Correta
onClick é acionado quando o usuário clica em um elemento.
- ✓ Q7 - Correta
this em funções construtoras refere-se ao objeto que está sendo criado.
- ✓ Q8 - Correta
Arrow functions NÃO têm seu próprio this, herdam do escopo pai.
- ✓ Q9 - Correta
O DOM representa a página como uma árvore de nós (elementos).
- ✓ Q10 - Correta
document.getElementById() retorna o elemento com o ID especificado.

👏 **Bom trabalho!** Você acertou 9/10. Revise os tópicos que errou!

📷 **IMPORTANTE:** Clique em "Imprimir" para salvar o resultado em PDF.